

halfedge: QEM 网格简化模块设计文档

src/decimate.rs

halfedge 库

2026-07-04

目录

1	模块定位	2
1.1	核心 API	2
1.2	依赖关系	2
2	Quadric 二次误差矩阵	2
2.1	数据结构	2
2.2	基本二次误差矩阵	2
2.3	顶点 Quadric 累加	3
2.4	evaluate 实现	3
3	最优折叠位置求解	3
3.1	目标	3
3.2	3×3 线性方程组	3
3.3	候选位置回退	4
4	算法流程	4
4.1	流程图	4
4.2	算法步骤详解	4
5	惰性删除策略	5
5.1	问题	5
5.2	方案	5
6	边界处理	6
7	复杂度分析	6
8	与 collapse_edge_at 的协作	6
9	退化健壮性	6
10	测试覆盖	7

1 模块定位

decimate.rs 在 topology_ops(collapse_edge_at)与 geometry(face_normal)之上叠加基于 **Quadric Error Metrics** 的网格简化能力。参考 Garland & Heckbert 1997 ”Surface Simplification Using Quadric Error Metrics”。

1.1 核心 API

API	语义
decimate_qem(&mut mesh, target_faces)	QEM 简化到目标面数，返回实际移除面数
decimate_to_vertices(&mut mesh, target_verts)	简化到目标顶点数（闭合流形 $F = 2V - 4$ ）

1.2 依赖关系

依赖模块	用途
storage	MeshStorage 容器
geometry	face_normal（面法向 → 平面方程）
topology_ops	collapse_edge_at（指定位置折叠）
traversal	is_boundary_edge、FaceHalfEdges、VertexRing

2 Quadric 二次误差矩阵

2.1 数据结构

4×4 对称矩阵，利用对称性存储 10 个独立分量：

```
1 #[derive(Clone, Debug)]
2 struct Quadric {
3     // [a^2, ab, ac, ad, b^2, bc, bd, c^2, cd, d^2]
4     //      :
5     // [ q00 q01 q02 q03 ]
6     // [ q01 q11 q12 q13 ]
7     // [ q02 q12 q22 q23 ]
8     // [ q03 q13 q23 q33 ]
9     data: [f64; 10],
10 }
```

2.2 基本二次误差矩阵

对平面方程 $ax + by + cz + d = 0$ ((a, b, c) 为**单位**法向)，基本二次误差矩阵：

$$K_p = \mathbf{p} \cdot \mathbf{p}^T, \quad \mathbf{p} = [a, b, c, d]$$

顶点 $\mathbf{v} = [x, y, z, 1]$ 到该平面的距离平方:

$$\Delta(\mathbf{v}) = \mathbf{v}^T K_p \mathbf{v} = (ax + by + cz + d)^2$$

2.3 顶点 Quadric 累加

顶点 v 的 Quadric 为其所有邻接面平面 K_p 之和:

$$Q_v = \sum_{f \in N(v)} K_{p_f}$$

评估顶点 v 的二次误差:

$$\Delta(v) = \mathbf{v}^T Q_v \mathbf{v} = \sum_{f \in N(v)} (a_f x + b_f y + c_f z + d_f)^2$$

即该顶点到所有邻接面平面距离平方之和。

2.4 evaluate 实现

```
1 fn evaluate(&self, pos: [f64; 3]) -> f64 {
2     let [x, y, z] = pos;
3     let q = &self.data;
4     q[0]*x*x + 2.0*q[1]*x*y + 2.0*q[2]*x*z + 2.0*q[3]*x
5     + q[4]*y*y + 2.0*q[5]*y*z + 2.0*q[6]*y
6     + q[7]*z*z + 2.0*q[8]*z
7     + q[9]
8 }
```

3 最优折叠位置求解

3.1 目标

折叠边 (v_0, v_1) 后, 合并 Quadric $Q = Q_{v_0} + Q_{v_1}$ 。最优新顶点位置:

$$\mathbf{p}^* = \arg \min_{\mathbf{v}} \mathbf{v}^T Q \mathbf{v}$$

3.2 3×3 线性方程组

令 $\partial(\mathbf{v}^T Q \mathbf{v})/\partial x = \partial/\partial y = \partial/\partial z = 0$, 得:

$$\begin{bmatrix} q_{00} & q_{01} & q_{02} \\ q_{01} & q_{11} & q_{12} \\ q_{02} & q_{12} & q_{22} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = - \begin{bmatrix} q_{03} \\ q_{13} \\ q_{23} \end{bmatrix}$$

用 Cramer 法则求解。奇异性判定使用相对阈值:

$$|\det A| < \varepsilon \cdot \|A\|_F^3, \quad \varepsilon = 10^{-12}$$

其中 $\|A\|_F$ 是 3×3 块的 Frobenius 范数。相对阈值在不同坐标量级 (小网格 $\det \sim 1$, 大网格 $\det \sim 10^{12}$) 下均能正确区分奇异与非奇异, 而绝对阈值 10^{-12} 在大网格上会把可解系统误判为奇异。 $\|A\|_F = 0$ (零矩阵) 直接判奇异。

3.3 候选位置回退

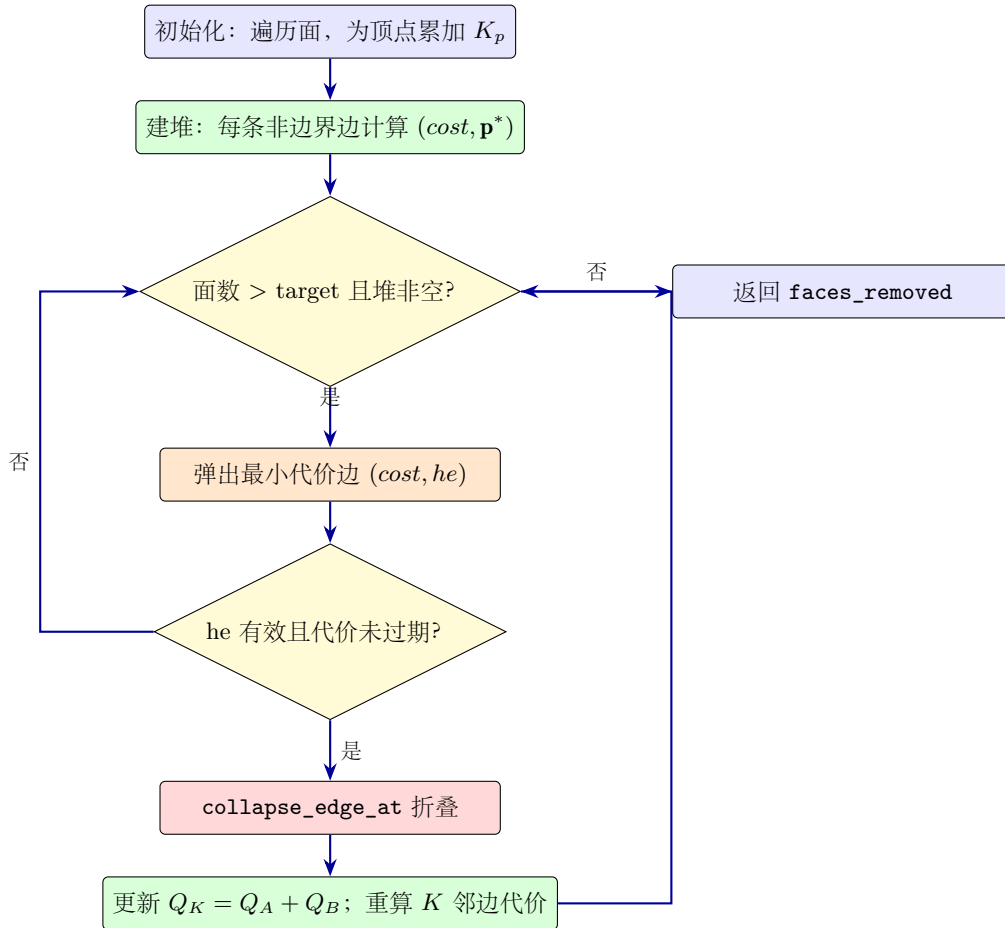
QEM 矩阵奇异时（如共面区域），最优位置不确定。本实现评估 4 个候选位置的代价，取最小者：

1. $\mathbf{p}^*$ （若 Cramer 法则有解）；
2. $\mathbf{v}_0$ （原始端点 A）；
3. $\mathbf{v}_1$ （原始端点 B）；
4. $\frac{\mathbf{v}_0 + \mathbf{v}_1}{2}$ （中点）。

这与 Garland-Heckbert 原文的建议一致，保证退化情况也能给出合理位置。

4 算法流程

4.1 流程图



4.2 算法步骤详解

步骤 1：初始化顶点 Quadric 遍历所有三角面 f ，计算平面方程 (a_f, b_f, c_f, d_f) （ a, b, c 为单位法向， $d = -\mathbf{n} \cdot \mathbf{v}_0$ ）。为面的每个顶点累加 K_{p_f} 。

步骤 2：构建边代价堆 遍历所有半边，跳过边界边。对每条边 (v_0, v_1) ：

- $Q = Q_{v_0} + Q_{v_1}$;
- $\mathbf{p}^* = \arg \min \mathbf{v}^T Q \mathbf{v}$ (Cramer 法则，奇异时回退);
- $cost = \min_{\mathbf{p} \in \{\mathbf{p}^*, \mathbf{v}_0, \mathbf{v}_1, \text{mid}\}} \mathbf{p}^T Q \mathbf{p}$;
- 压入最小堆 ($Reverse(cost), he$)。

步骤 3：贪心折叠循环

1. 弹出最小代价边 ($cost, he$);
2. 惰性删除检测：若 he 已删除或 $cost$ 与 $cost_map$ 中存储的不符，跳过;
3. 检查是否已变为边界边，若是跳过;
4. 收集 6 条待删除半边 $[he, twin, n_1, p_1, n_2, p_2]$;
5. 调用 `collapse_edge_at(mesh, he,  $\mathbf{p}^*$ )`;
6. 更新 $Q_K = Q_A + Q_B$;
7. 清理被删除半边的 $cost_map$ 条目;
8. 对 K 的每条邻接边重算 ($cost, \mathbf{p}^*$)，压入堆并更新 $cost_map$ 。

5 惰性删除策略

5.1 问题

折叠一条边后，与 A 、 B 相邻的其他边的 Quadric 已改变（因 $Q_K \neq Q_A$ ），堆中这些边的旧代价已过期。直接从堆中删除任意元素是 $O(n)$ 的。

5.2 方案

采用**惰性删除** (lazy deletion)：

- 维护 `HashMap<HalfEdgeId, (f64, [f64;3])>` 记录每个半边的**最新**代价和位置;
- 弹出堆顶 ($heap_cost, he$) 后，比较 $heap_cost$ 与 $cost_map[he]$ 的值;
- 差值 $> 10^{-9} \cdot \max(|heap_cost|, |stored_cost|, 1)$ 则为过期条目，跳过（**相对容差**：绝对阈值在大坐标网格上 $cost \sim 10^6$ 时会把合法条目误判为过期）;
- 折叠后重新计算 K 的邻边代价，更新 $cost_map$ 并压入新条目。

过期条目占用的内存会在后续弹出时自动释放。每个条目最多被压入堆 $O(valence)$ 次，总堆操作 $O(E \cdot \bar{v} \cdot \log E)$ 。

6 边界处理

情况	处理
边界边	不加入堆，不参与折叠（保持边界拓扑）
QEM 矩阵奇异（共面区域）	回退到 4 候选位置中最小代价者
折叠失败（链接条件、退化）	跳过该边，继续弹出下一条
<code>target_faces</code> $\geq$ 当前面数	直接返回 <code>Ok(0)</code> ，不操作
堆空且面数仍 $>$ <code>target</code>	返回已移除面数（可能未达 <code>target</code> ）

7 复杂度分析

设原始网格有 V 顶点、 E 边、 F 面，目标移除 k 条边 ($k \leq E$)。

步骤	时间	空间
初始化顶点 Quadric	$O(F)$	$O(V)$
建堆（每条边一次）	$O(E \log E)$	$O(E)$
单次折叠（含邻边重算）	$O(\bar{v} \log E)$	$O(\bar{v})$
k 次折叠总计	$O(k\bar{v} \log E)$	$O(E)$
总计	$O((F + k\bar{v}) \log E)$	$O(V + E)$

其中 $\bar{v}$ 为平均顶点 valence（通常 6）。对于简化到 50%， $k \approx E/4$ ，总计 $O(E \log E)$ 。

8 与 `collapse_edge_at` 的协作

`decimate.rs` 不直接操作半边拓扑，而是通过 `collapse_edge_at` 委托给 `topology_ops` 模块。调用形式为 `collapse_edge_at(mesh, he, p*)`，其中 p^* 为 QEM 求出的最优折叠位置。该函数：

- 校验链接条件（防止非流形）；
- 创建新顶点 K 位置为 p^* （QEM 最优位置，而非中点）；
- 缝合 twin、更新 vertex、删除 6 条半边 + 2 面 + 2 顶点；
- 返回 K 的 `VertexId`。

9 退化健壮性

- 除零：Cramer 法则检查 $|\det| < 10^{-12} \cdot \|A\|_F^3$ （相对阈值）返回 `None`；
- NaN 代价：`edge_cost_and_position` 中将 NaN 转为 $+\infty$ ；
- 无效 ID：所有 `mesh.get_*` 返回 `None` 时跳过；

- 空网格: `face_count() == 0` 时 `target ≥ 0` 直接返回;
- `CostKey Ord`: `partial_cmp().unwrap_or(Equal)` 处理 NaN 比较不 panic。

10 测试覆盖

测试名	验证内容
<i>Quadric</i> 单元测试	
<code>quadric_zero_evaluates_to_zero</code>	零矩阵任意点误差 = 0
<code>quadric_from_plane_zero_at_plane_point</code>	平面上的点误差 = 0, 平面外 = 距离 ²
<code>quadric_add_is_commutative</code>	$Q_1 + Q_2 = Q_2 + Q_1$
<code>quadric_find_optimal_position_singular_returns_none</code>	秩 1 矩阵返回 None
<code>quadric_find_optimal_position_two_planes</code>	两正交平面仍奇异 (z 自由)
<code>quadric_find_optimal_position_three_planes</code>	三正交平面最优 = 原点
<i>decimate_gem</i> 集成测试	
<code>decimate_icosphere2_to_80_faces</code>	<code>icosphere(2)</code> $F=320 \rightarrow \leq 80$, 校验通过
<code>decimate_icosphere2_to_tetrahedron</code>	简化到 ≤ 8 面, 仍为有效闭合格
<code>decimate_flat_plane_all_zero_cost</code>	平面所有代价 = 0, 正确折叠对角线
<code>decimate_boundary_edges_not_collapsed</code>	单三角形全边界边, 移除 = 0
<code>decimate_target_geq_current_is_noop</code>	<code>target ≥</code> 当前面数时不操作
<code>decimate_icosphere1_half_simplification</code>	<code>icosphere(1)</code> $F=80 \rightarrow \leq 40$, 校验通过
<code>decimate_preserves_closed_topology</code>	闭合格 Euler 示性数 = 2 保持
<code>decimate_to_vertices_icosphere2</code>	目标 42 顶点 $\rightarrow \leq 80$ 面
<code>decimate_multiple_iterations_stay_valid</code>	连续 3 次简化均通过完整校验

共 15 个单元测试, 全部通过。全库 `cargo test` 共 371 个用例, 0 失败。

11 设计取舍

- 为何用 Cramer 法则而非 Gaussian 消元? 3×3 系统的 Cramer 法则只需 1 个行列式 + 3 个余子式, 代码简洁且无内存分配。对更高维系统应改用 LU 分解。
- 为何评估 4 个候选位置而非仅 $\mathbf{p}^*$? 共面区域的 Q 矩阵奇异 (秩 < 3), $\mathbf{p}^*$ 无解。回退到端点/中点保证退化情况也有合理位置。Garland-Heckbert 原文也建议此策略。
- 为何用惰性删除而非 Fibonacci 堆? `BinaryHeap` 是标准库自带的最小堆实现 (配合 `Reverse`), 惰性删除避免 $O(n)$ 的堆内查找。总堆操作 $O(k\bar{v} \log E)$ 在实践中足够快 (`icosphere(3)` 简化 $50\% < 1$ 秒)。
- 为何不实现边界约束 Quadric? 原文建议为边界边添加约束平面 (两个垂直于边界的平面) 以保持锐边。本实现简单地跳过边界边, 已能满足「边界不退化」的需求。边界约束可作为后续增强。

- 为何 `decimate_to_vertices` 用 $F = 2V - 4$? 闭合三角网格满足 Euler 公式 $V - E + F = 2$ 且 $E = 3F/2$, 故 $V = F/2 + 2$, 即 $F = 2(V - 2) = 2V - 4$ 。对带边界网格此公式不精确, 但作为目标估计已足够。